

RTL Design Sherpa

Delta Micro-Architecture Specification 1.3

October 20, 2026

Table of Contents

1 Delta Mas Index....11

2 Delta Hardware Architecture Specification Index....12

2.1 Overview....12

2.2 Related Modules....13

2.3 Navigation....14

2.3.1 Front Matter....14

2.3.2 Chapter 1: Introduction....14

2.3.3 Chapter 2: System Overview....14

2.3.4 Chapter 3: Architecture....14

2.3.5 Chapter 4: Interfaces....14

2.3.6 Chapter 5: Performance....14

2.3.7 Chapter 6: Integration....14

2.3.8 Chapter 7: Mesh Network....15

3 Document Information....16

3.1 Delta Micro-Architecture Specification....16

3.2 Revision History....17

3.3 Document Purpose....18

3.4 Intended Audience....19

3.5 Related Documents....20

4 Overview....21

4.1 What This Document Describes....21

4.2 Generated Module Structure....22

4.3 Key Micro-Architectural Decisions....23
4.4 Block Diagram....24
4.4.1 Figure 1.1: Delta Micro-Architecture Block Diagram....24
4.5 Reference RTL....25
5 Request Decode....26
5.1 Purpose....26
5.2 Implementation....27
5.3 TDEST Range Check....28
5.4 Relationship to Arbitration....29
5.5 Notes....30
6 Per-Slave Round-Robin Arbiter....31
6.1 State Variables....31
6.2 FSM Behavior....32
6.2.1 Idle State (packet_active[s] == 0)....32
6.2.2 Active State (packet_active[s] == 1)....32
6.3 Round-Robin Pseudocode....33
6.4 Fairness....34
6.5 Reset Behavior....35
7 Crossbar Multiplexer....36
7.1 Purpose....36
7.2 Implementation....37
7.3 Signal Selection....38
7.4 Default Values....39
7.5 Timing....40

8 TREADY Steering..... 41
8.1 Purpose..... 41
8.2 Implementation..... 42
8.3 Behavior..... 43
8.4 Deadlock Avoidance..... 44
9 TWAKEUP Steering (AMBA5)..... 45
9.1 Purpose..... 45
9.2 Implementation..... 46
9.3 Semantics..... 47
9.4 Integration Notes..... 48
10 Interface Check Signals (AMBA5)..... 49
10.1 Purpose..... 49
10.2 Parity Rule..... 50
10.3 Implementation..... 51
10.4 Design Notes..... 52
10.5 Verification..... 53
11 Packet Counters..... 54
11.1 Purpose..... 54
11.2 Implementation..... 55
11.3 Behavior..... 56
11.4 Disabling Counters..... 57
12 Assertions..... 58
12.1 Overview..... 58
12.2 One-Hot Grant Check..... 59

12.3 TDEST Range Check 60
12.4 Enabling Assertions 61
12.5 Notes 62
13 Multi-Master Contention 63
13.1 Scenario 63
13.2 Cycle Trace 64
13.3 Observations 66
13.4 Same-Cycle Release and Re-Arbitration 67
14 Packet Interleave Prevention 68
14.1 Rule 68
14.2 Mechanism 69
14.3 Why It Matters 70
14.4 Example: Attempted Interleave 71
14.5 Corner Cases 72
15 Module Structure 73
15.1 Generated File 73
15.2 Module Header 74
15.2.1 Base Flavor Header 74
15.2.2 Rich Flavor Additions 74
15.3 Internal Organization 75
15.4 Filelist 76
16 Parameters and Logic Implications 77
16.1 Parameter Summary 77
16.2 Width Calculations 78

16.3 Parameter Interaction....79
16.4 RTL Constants....80
17 Test Strategy....81
17.1 Current Status....81
17.2 Recommended Test Plan....82
17.2.1 1. Routing Coverage....82
17.2.2 2. Arbitration Fairness....82
17.2.3 3. Packet Atomicity....82
17.2.4 4. Backpressure....82
17.2.5 5. Reset....82
17.2.6 6. Rich Flavor....82
17.2.7 7. Formal Assertions....82
17.3 Lint and Generation Checks....84
18 Buffers and Virtual Channels....85
18.1 Per-Port Input Buffer Organization....86
18.2 Why Per-VC Reservation Is Required....87
18.3 Credit-Based Flow Control....88
18.4 Failure Mode: Ready Drops Mid-Burst....89
18.5 References....90
19 Pipeline Stages....91
19.0.1 Figure 6.1: delta_mesh_cell internal data path....91
19.0.2 Figure 6.1: delta_mesh_cell internal data path....91
19.1 Route Lookahead....93
19.2 Speculative VC and Switch Allocation....94

19.3 Handshake Discipline..... 95
19.4 Failure Mode: TLAST Arrives Without a Matching Grant.....96
19.5 References.....97
20 Routing Table..... 98
20.1 RAM Table Structure.....99
20.2 Reset Image.....100
20.3 In-Band Load and Readback.....101
20.4 VC0 Acyclicity Check.....102
20.5 Torus Wrap and the Dateline Rule.....103
20.6 Standard Fabric Modes.....104
20.7 References.....105
21 Allocators.....106
21.1 VC Allocation.....107
21.2 Switch Allocation.....108
21.3 One-Hot Grant Assertions.....109
21.4 Multicast Allocation.....110
21.5 Conservative Output-VC Reallocation.....111
21.6 References.....112
22 QoS Mechanisms.....113
22.1 Strict Priority and Programmable Aging.....114
22.2 Bulk Minimum Reservation.....115
22.3 Class Non-Interference.....116
22.4 Failure Mode: Aging Inversion Misconfiguration.....117
22.5 References.....118

23 Multicast Path..... 119
23.1 Source-Side Replication..... 120
23.2 Delivery-VC Assignment..... 121
23.3 The No-Responses Rule..... 122
23.4 Failure Mode: Tree-Based Multicast in Wormhole Networks..... 123
23.5 References..... 124
24 Observability Counters and DFT..... 125
24.1 Ocelus Counter Bank..... 126
24.2 DFT Test Mode..... 127
24.3 References..... 128
25 Verification Targets..... 129
25.1 cocotb Suites..... 130
25.2 Formal Assists..... 132
25.3 Performance Model Calibration..... 133
25.4 References..... 134

List of Figures

Figure 1.1: Delta Micro-Architecture Block Diagram.....24
Figure 6.1: delta_mesh_cell internal data path.....90

List of Tables

Table 1: Document Properties....16
Table 2: Revision History....17
Table 3: Key Micro-Architectural Decisions....23
Table 4: Arbiter State Variables....31
Table 5: Contention Trace....64
Table 6: Interleave Prevention....71
Table 7: Parameter-to-Logic Mapping....77
Table 8: Input buffer organization....85
Table 9: Credit counter sizing....87
Table 10: Cell pipeline stages....90
Table 11: Routing table entry layout....98
Table 12: Standard fabric mode catalog....103
Table 13: Allocator mapping....105
Table 14: QoS class mapping....112
Table 15: Class isolation resources....115
Table 16: Multicast discipline....121
Table 17: Ocelus counter bank indices....125
Table 18: DFT routing patterns....126
Table 19: Mesh cocotb verification suites....129

1 Delta Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Delta Hardware Architecture Specification Index

2.1 Overview

Version: 1.3 **Date:** 2026-10-20 **Purpose:** High-level hardware architecture specification for the Delta AXI4-Stream crossbar generator, including the planned mesh network mode

---

2.2 Related Modules

- **Delta MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide
- **QUICK_START.md** - Hands-on generator guide

2.3 Navigation

Note: Every chapter below is one source file; the document build assembles the spec from these links.

2.3.1 Front Matter

- Document Information

2.3.2 Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

2.3.3 Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

2.3.4 Chapter 3: Architecture

- Block Diagram
- Data Flow
- Protocol Support

2.3.5 Chapter 4: Interfaces

- AXI-Stream Base Interface
- AXI-Stream Rich Interface (TSTRB + AMBA5)
- Clock and Reset

2.3.6 Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resources

2.3.7 Chapter 6: Integration

- System Requirements
- Parameter Configuration
- Verification Strategy
- Synthesis and Implementation

2.3.8 Chapter 7: Mesh Network

- Hierarchy
- Cell and Flit
- Routing and Configuration
- Flow Control and Deadlock
- Quality of Service
- Multicast
- Systolic Support and Fabric Modes
- Observability and DFT
- Performance and Verification
- Deferred Items
- Resilient Routing: Open Hazards

Last Updated: 2026-10-14 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Document Information

3.1 Delta Micro-Architecture Specification

Document Properties

Property	Value
Document Title	Delta Micro-Architecture Specification
Version	1.3
Date	2026-10-20
Status	Released
Classification	Open Source - MIT License

3.2 Revision History

Revision History

Version	Date	Author	Description
1.0	2026-10-12	RTL Design Sherpa	Initial release; describes the generated flat crossbar RTL including request decode, packet-atomic RR arbitration, mux, TREADY/TWAKEUP steering, counters, and assertions.
1.1	2026-10-12	RTL Design Sherpa	AMBA 5 interface parity per IHI 0051B ch. 5 (issue #102): `TPARITY`/`PW_WIDTH`/`--parity-width` replaced by the odd-parity check signals `TDATACHK` … `TWAKEUPCHK` (spec widths, checked at every port, sticky `*_parity_error`), matching `rtl/amba/axis5`. First `dv/` suite.
1.2	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
1.3	2026-10-20	RTL Design Sherpa	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

3.3 Document Purpose

This Micro-Architecture Specification (MAS) is the internals document for the Delta AXI4-Stream crossbar generator — the one you open when you need to know what the generated RTL actually does, block by block. It covers:

- Block-level implementation of the generated SystemVerilog
- Arbitration state behavior and grant hold/release rules
- Cycle-level behavior examples
- Parameter-to-logic implications
- Verification strategy

For the high-level architecture, interface requirements, and integration guidelines, see the companion **Delta Hardware Architecture Specification (HAS)**.

3.4 Intended Audience

- RTL engineers reading or modifying the generated crossbar
- Verification engineers writing protocol checks
- System architects resolving timing or arbitration questions
- Technical reviewers auditing the implementation

3.5 Related Documents

- **Delta HAS** - Hardware Architecture Specification (high-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI assistant integration guide
- **QUICK_START.md** - Hands-on generator guide

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

4 Overview

4.1 What This Document Describes

This Micro-Architecture Specification (MAS) describes the internals of the SystemVerilog module produced by `bin/delta_generator.py`. It is intended for readers who need to understand, debug, or modify the generated RTL.

4.2 Generated Module Structure

The generator emits a single module named `delta_axis_flat_MxN` where `M` is `NUM_MASTERS` and `N` is `NUM_SLAVES`. The module contains:

1. **TDEST decode** — builds the per-slave request matrix.
2. **Per-slave round-robin arbiter** — packet-atomic grant logic.
3. **Crossbar mux** — combinational selection of payload and control.
4. **TREADY steering** — backpressure routing.
5. **TWAKEUP steering** — AMBA 5 wakeup hint routing (optional).
6. **Packet counters** — per-master and per-slave counters (optional).
7. **Assertions** — `ifdef FORMAL` checks.

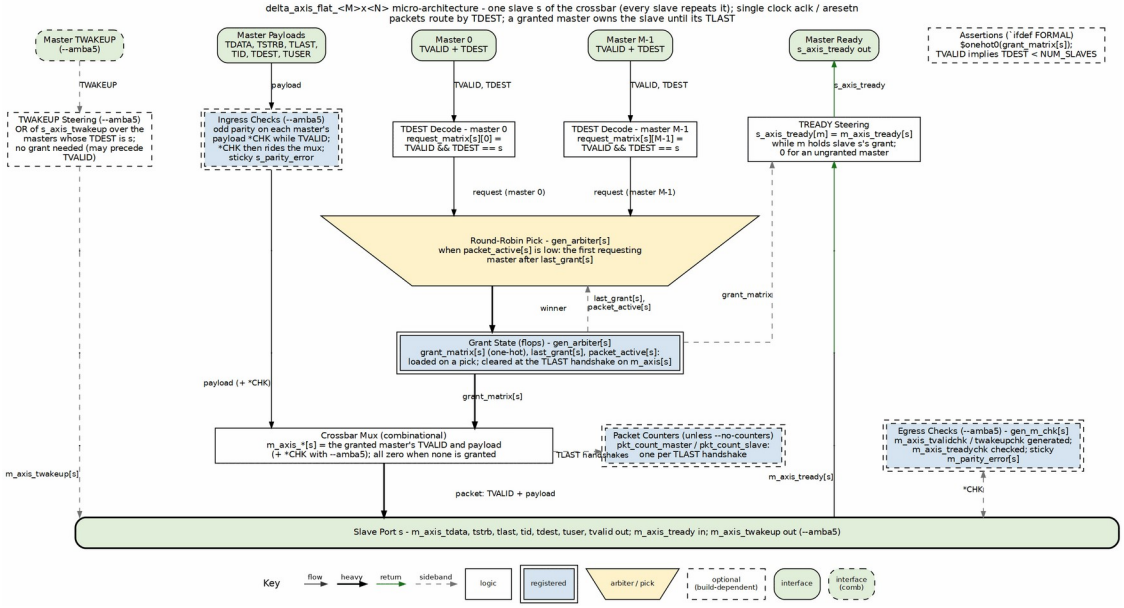
4.3 Key Micro-Architectural Decisions

Key Micro-Architectural Decisions

Decision	Rationale
Registered grant, combinational mux	Two-cycle latency with a short critical path.
Per-slave arbiter	Independent scheduling; no head-of-line blocking across slaves.
TDEST direct decode	No address map; simpler and lower latency than address-range decode.
Packet-atomic grant	Preserves AXI4-Stream packet semantics; prevents interleaving.
Optional sidebands default off	Keeps base output byte-identical to earlier versions.

4.4 Block Diagram

4.4.1 Figure 1.1: Delta Micro-Architecture Block Diagram



Delta MAS Block Diagram

The diagram follows one slave `s` of the generated RTL; every slave repeats it. Each master's TVALID and TDEST become one bit of `request_matrix[s]`; the round-robin pick (the trapezium) chooses the first requester after `last_grant[s]` while no packet is active; the grant-state flops hold `grant_matrix[s]` until the TLAST handshake; and the combinational mux forwards the granted master's TVALID and payload, which arrive on their own lane at the left. TREADY climbs the right lane to the granted master only. Dashed blocks are build options: TWAKEUP steering and the ingress and egress `*CHK` checks with `--amba5`, the packet counters unless `--no-counters`, and the assertions under `` `ifdef FORMAL ``.

4.5 Reference RTL

The descriptions in this document are verified against:

- `rtl/delta_axis_flat_4x16.sv` (base flavor)
- `rtl_test/delta_axis_flat_2x4_strb_amba5.sv` (rich flavor)

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

5 Request Decode

5.1 Purpose

The request decode block translates each master's `s_axis_tvalid` and `s_axis_tdest` into a per-slave request matrix. A set bit in `request_matrix[s][m]` indicates that master `m` is requesting slave `s`.

5.2 Implementation

The decode is combinational:

```
logic [NUM_MASTERS-1:0] request_matrix [NUM_SLAVES];

always_comb begin
    for (int s = 0; s < NUM_SLAVES; s++) begin
        request_matrix[s] = '0;
    end
    for (int m = 0; m < NUM_MASTERS; m++) begin
        if (s_axis_tvalid[m] &&
            (s_axis_tdest[m] < NUM_SLAVES[$clog2(NUM_SLAVES+1)-1:0]))
begin
            request_matrix[s_axis_tdest[m]][m] = 1'b1;
        end
    end
end
```

5.3 TDEST Range Check

The comparison `s_axis_tdest[m] < NUM_SLAVES` prevents out-of-range indices from corrupting the `request_matrix` array. The width of the comparison is sized to cover the full `NUM_SLAVES` value.

5.4 Relationship to Arbitration

The request matrix is the input to each slave's round-robin arbiter. Only slaves with at least one request bit set are candidates for a new grant.

5.5 Notes

- The decode does not depend on `TREADY`; a master may request even while stalled.
- `TDEST` is sampled combinationally and must be stable while `TVALID` is asserted.
- There is no address map; `TDEST` is the slave index directly.

6 Per-Slave Round-Robin Arbiter

6.1 State Variables

Each slave `s` maintains three registered variables:

Arbiter State Variables

Variable	Width	Purpose
`grant_matrix[s]`	`NUM_MASTERS` bits	One-hot (or zero-hot) grant vector.
`last_grant[s]`	`clog2(NUM_MASTERS)` bits	Pointer to the most recently granted master.
`packet_active[s]`	1 bit	Set while a packet is in progress on this slave.

6.2 FSM Behavior

The arbiter has two operational states for each slave:

6.2.1 Idle State (`packet_active[s] == 0`)

- If `request_matrix[s]` is non-zero, perform round-robin arbitration.
- The search starts at `last_grant[s] + 1` and wraps through all masters.
- The first requesting master is granted; `last_grant[s]` is updated and `packet_active[s]` is set.
- If no master requests, `grant_matrix[s]` remains zero.

6.2.2 Active State (`packet_active[s] == 1`)

- The current grant is held.
- On the cycle where `m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]` is true, `packet_active[s]` is cleared and `grant_matrix[s]` is reset to zero.
- If `TLAST` is accepted, the next cycle may re-arbitrate for a new master.

6.3 Round-Robin Pseudocode

```
if (packet_active[s]) begin
    if (m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]) begin
        packet_active[s] <= 1'b0;
        grant_matrix[s]  <= '0;
    end
end else if (|request_matrix[s]) begin
    grant_matrix[s] = '0;
    for (int i = 0; i < NUM_MASTERS; i++) begin
        int m = (last_grant[s] + 1 + i) % NUM_MASTERS;
        if (request_matrix[s][m] && grant_matrix[s] == '0) begin
            grant_matrix[s][m] = 1'b1;
            last_grant[s]      = m[$clog2(NUM_MASTERS)-1:0];
            packet_active[s]   = 1'b1;
        end
    end
end else begin
    grant_matrix[s] <= '0;
end
```

6.4 Fairness

Round-robin guarantees that no requesting master is starved. The longest wait for a contending master is bounded by the number of other contending masters and the length of their packets.

6.5 Reset Behavior

On reset:

- `grant_matrix[s] <= '0`
- `last_grant[s] <= '0`
- `packet_active[s] <= 1'b0`

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

7 Crossbar Multiplexer

7.1 Purpose

The crossbar mux selects the payload and control signals from the granted master and drives them onto the selected slave interface. There is one mux per slave.

7.2 Implementation

Each slave mux is combinational:

```
generate
    for (genvar s = 0; s < NUM_SLAVES; s++) begin : gen_slave_mux
        always_comb begin
            m_axis_tdata[s]  = '0;
            m_axis_tvalid[s] = 1'b0;
            m_axis_tlast[s]  = 1'b0;
            m_axis_tdest[s]  = '0;
            m_axis_tid[s]    = '0;
            m_axis_tuser[s]  = '0;

            for (int m = 0; m < NUM_MASTERS; m++) begin
                if (grant_matrix[s][m]) begin
                    m_axis_tdata[s]  = s_axis_tdata[m];
                    m_axis_tvalid[s] = s_axis_tvalid[m];
                    m_axis_tlast[s]  = s_axis_tlast[m];
                    m_axis_tdest[s]  = s_axis_tdest[m];
                    m_axis_tid[s]    = s_axis_tid[m];
                    m_axis_tuser[s]  = s_axis_tuser[m];
                end
            end
        end
    end
endgenerate
```

7.3 Signal Selection

The mux forwards the following signals from the granted master to the selected slave:

- `TDATA`
- `TVALID`
- `TLAST`
- `TDEST`
- `TID`
- `TUSER`
- `TSTRB` (when enabled)
- The payload check bits `TDATACHK`, `TSTRBCHK`, `TLASTCHK`, `TIDCHK`, `TDESTCHK`, `TUSERCHK` (with `--amba5`). They are carried unchanged, not regenerated. See Interface Check Signals (AMBA5).

7.4 Default Values

When no grant is active for a slave, the mux drives all outputs to zero and `m_axis_tvalid[s]` to low. This is a safe idle state. With `--amba5`, the payload check outputs idle at all ones instead, which is the odd parity of the all-zero idle payload.

7.5 Timing

Because the mux is combinational, the output settles after the grant and the input signals are stable. The grant is registered, so the mux select is stable for a full clock cycle once a packet begins.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

8 TREADY Steering

8.1 Purpose

TREADY steering routes each slave's `m_axis_tready` back to the master that currently holds the grant for that slave. Ungranted masters see `s_axis_tready[m] == 0`.

8.2 Implementation

```
generate
    for (genvar m = 0; m < NUM_MASTERS; m++) begin : gen_master_tready
        always_comb begin
            s_axis_tready[m] = 1'b0;
            for (int s = 0; s < NUM_SLAVES; s++) begin
                if (grant_matrix[s][m]) begin
                    s_axis_tready[m] = m_axis_tready[s];
                end
            end
        end
    end
endgenerate
```

8.3 Behavior

- Only one slave can grant a given master at a time (packets are atomic and a master targets one slave per packet), so the loop effectively selects a single `m_axis_tready[s]`.
- If a master is not currently granted by any slave, it is stalled.
- Backpressure propagates combinationally from slave to master.

8.4 Deadlock Avoidance

The design avoids deadlock because:

- Grants are released after TLAST, freeing the slave for new requests.
- TREADY is combinationally steered; a slave ready transition immediately reaches the granted master.
- There are no FIFOs or credit loops in the base crossbar.

9 TWAKEUP Steering (AMBA5)

9.1 Purpose

When `--amba5` is enabled, the generated module includes `s_axis_twakeup` and `m_axis_twakeup` ports. `TWAKEUP` is an AMBA 5 hint that a master will soon send a transaction; it may assert before `TVALID`.

9.2 Implementation

The wakeup steering is combinational and independent of the grant:

```
always_comb begin
    for (int s = 0; s < NUM_SLAVES; s++) begin
        m_axis_twakeup[s] = 1'b0;
        for (int m = 0; m < NUM_MASTERS; m++) begin
            if (s_axis_twakeup[m] &&
                (s_axis_tdest[m] == s[$clog2(NUM_SLAVES)-1:0])) begin
                m_axis_twakeup[s] = 1'b1;
            end
        end
    end
end
```

9.3 Semantics

- `m_axis_twakeup[s]` is the OR of `s_axis_twakeup[m]` from every master whose `TDEST` points to slave `s`.
- No grant is required; the hint is decoded directly from `TDEST`.
- `TWAKEUP` may be high while `TVALID` is low.
- The hint is not a handshake; the slave is not required to respond before data arrives.

9.4 Integration Notes

- `TWAKEUP` is only emitted when `--amba5` is specified.
- `--amba5` also emits `TWAKEUPCHK`. The crossbar generates `m_axis_twakeupchk[s]` from this OR and checks `s_axis_twakeupchk[m]`. See Interface Check Signals (AMBA5).
- `TWAKEUP` does not affect arbitration timing.

10 Interface Check Signals (AMBA5)

10.1 Purpose

When `--amba5` is enabled, the generated module carries the AXI5-Stream interface-protection signals of IHI 0051B chapter 5 (`Check_Type = Odd_Parity_Byte_All`) and checks them at its own ports. This is the `generate_check_logic()` section of `bin/delta_generator.py`. Names and widths match `rtl/amba/axis5/*`.

10.2 Parity Rule

Odd parity: a check bit and the bits it covers hold an odd number of ones. Check bit `[n]` covers bits `[8n+7:8n]`, and the top bit covers a short remainder (section 5.3). A one-bit check (`TLASTCHK`, `TVALIDCHK`, `TREADYCHK`, `TWAKEUPCHK`) is the inversion of its signal. Every check bit is one `rtl/common/dataint_parity` instance with `CHUNKS=1` and `parity_type = 1'b0` (odd), so the spec's per-byte layout holds at any width.

10.3 Implementation

Two generate loops follow the TWAKEUP steering block.

Per master port (`gen_s_chk[m]`). The crossbar is the receiver here.

```
for (genvar g = 0; g < TDATACHK_W; g++) begin : g_tdatachk
    localparam int LO = 8 * g;
    localparam int PIECE_W = (DATA_WIDTH - LO < 8) ? (DATA_WIDTH - LO)
: 8;
    dataint_parity #(.CHUNKS(1), .WIDTH(PIECE_W)) u_par (
        .data_in     (s_axis_tdata[m][LO +: PIECE_W]),
        .parity_in   (s_axis_tdatachk[m][g]),
        .parity_type (1'b0),
        .parity      (w_tdatachk_exp[g]),
        .parity_err  (w_tdatachk_err[g])
    );
end
// ... the same for TSTRB (with --strb-width), TID, TDEST, TUSER;
TLAST is 1 bit
assign w_payload_err = s_axis_tvalid[m] && (|w_tdatachk_err || ...);
```

- The payload checks (`TDATACHK`, `TSTRBCHK`, `TLASTCHK`, `TIDCHK`, `TDESTCHK`, `TUSERCHK`) are enabled by `TVALID`.
- `TVALIDCHK` and `TWAKEUPCHK` are checked on every cycle out of reset.
- `s_axis_treadychk[m]` is generated: the `parity` output of a 1-bit `dataint_parity` over the `s_axis_tready[m]` the crossbar drives.
- Any failure sets `s_parity_error[m]`, a sticky flop that only `aresetn` clears.

Per slave port (`gen_m_chk[s]`). The crossbar is the transmitter here.

- `m_axis_tvalidchk[s]` and `m_axis_twakeupchk[s]` are generated from the `TVALID` / `TWAKEUP` the crossbar drives.
- `m_axis_treadychk[s]` is checked on every cycle out of reset. A failure sets the sticky `m_parity_error[s]`.

Carry. The payload check bits are not recomputed at the slave port. They travel through the crossbar mux (see Crossbar Multiplexer) with the same grant as `TDATA`. While no master is granted they idle at all ones, which is the odd parity of the all-zero idle payload.

10.4 Design Notes

- Carrying the check bits keeps protection end to end. A beat corrupted upstream of the crossbar still fails at the downstream receiver. If the crossbar regenerated the check bits from its own mux output, the fault would be hidden.
- A failed check does not stall or drop the beat. The fabric's job is to report the fault, and the system decides how to respond.
- `TWAKEUPCHK` on the slave port is generated from the OR in the TWAKEUP steering block, so it is valid before any grant.

10.5 Verification

`dv/tests/test_delta_axis_flat_2x4_strb_amba5.py` (routing, wakeup and chk_errors) exercises every row above. A mutation check of the generated RTL showed the suite failing in each of these cases: even parity instead of odd, the ingress payload check removed, `TDATACHK` corrupted on the carry path, and `TREADYCHK` not inverted.

---

11 Packet Counters

11.1 Purpose

Optional 32-bit packet counters track completed packets at the master boundary and the slave boundary. They are enabled by default and can be disabled with `--no-counters`.

11.2 Implementation

```
logic [31:0] pkt_count_master [NUM_MASTERS];
logic [31:0] pkt_count_slave [NUM_SLAVES];

generate
    for (genvar m = 0; m < NUM_MASTERS; m++) begin :
gen_master_counters
        always_ff @(posedge aclk or negedge aresetn) begin
            if (!aresetn)
                pkt_count_master[m] <= '0;
            else if (s_axis_tvalid[m] && s_axis_tready[m] &&
s_axis_tlast[m])
                pkt_count_master[m] <= pkt_count_master[m] + 1;
        end
    end

    for (genvar s = 0; s < NUM_SLAVES; s++) begin : gen_slave_counters
        always_ff @(posedge aclk or negedge aresetn) begin
            if (!aresetn)
                pkt_count_slave[s] <= '0;
            else if (m_axis_tvalid[s] && m_axis_tready[s] &&
m_axis_tlast[s])
                pkt_count_slave[s] <= pkt_count_slave[s] + 1;
        end
    end
endgenerate
```

11.3 Behavior

- A master counter increments when the master completes a packet (last beat accepted).
- A slave counter increments when the slave completes a packet (last beat accepted).
- Counters wrap at 32 bits.
- Counters are not exposed on module ports; they are internal observation points.

11.4 Disabling Counters

Use `--no-counters` to remove the counter logic and save area:

```
python3 bin/delta_generator.py --topology flat --masters 4 --slaves 16
\
    --data-width 64 --no-counters --output-dir rtl/
```

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

12 Assertions

12.1 Overview

All generated Delta modules include an `ifdef FORMAL` block with SystemVerilog assertions for debug and formal verification.

12.2 One-Hot Grant Check

Each slave must grant at most one master at a time:

```
assert property (@(posedge aclk) disable iff (!aresetn)
    $onehot0(grant_matrix[s])
) else $error("Grant matrix not one-hot for slave %0d", s);
```

`$onehot0` allows the grant vector to be all-zero (idle) but not multi-hot.

12.3 TDEST Range Check

Each master must drive a valid slave index when it asserts TVALID:

```
assert property (@(posedge aclk) disable iff (!aresetn)
    s_axis_tvalid[m] |-> s_axis_tdest[m] < NUM_SLAVES
) else $error("TDEST out of range for master %0d", m);
```

12.4 Enabling Assertions

Define `FORMAL` during simulation or formal analysis:

```
verilator --lint-only -DFORMAL rtl/delta_axis_flat_4x16.sv
```

12.5 Notes

- Assertions are disabled by default to avoid simulation overhead.
- The TDEST range assertion duplicates the combinational safety check in the request decode block.
- Additional protocol assertions (TVALID stability, TLAST packet integrity) are candidates for the planned cocotb testbench.

13 Multi-Master Contention

13.1 Scenario

Two masters, M0 and M1, both want to send packets to the same slave S0. S0 is initially idle. M0 sends a two-beat packet; M1 sends a one-beat packet.

13.2 Cycle Trace

Assumption: `last_grant[S0]` was 1 before the trace begins.

Contention Trace

Cycle	Event	request_matrix[S0]	grant_matrix[S0]	packet_active[S0]	Notes
0	M0 and M1 both request S0	2'b11	2'b00	0	Slave idle.
1	Arbiter picks M0 (RR after last_grant=1)	2'b11	2'b01	1	Grant registered.
2	M0 beat 0 accepted	2'b10	2'b01	1	M1 waits.
3	M0 beat 1 (TLAST) accepted	2'b10	2'b00	0	Packet ends; grant released.

Cycle	Event	request_matrix[S0]	grant_matrix[S0]	packet_active[S0]	Notes
4	Arbiter picks M1	2'b10	2'b10	1	M1 now granted.
5	M1 beat 0 (TLAST) accepted	2'b00	2'b00	0	Both packets complete.

13.3 Observations

- M1 waits while M0 completes its packet.
- The arbiter does not re-arbitrate inside a packet.
- After TLAST, the next arbitration cycle grants M1 because M0 was the previous grant.
- Round-robin ensures M1 is not starved.

13.4 Same-Cycle Release and Re-Arbitration

On the cycle that TLAST is accepted, the grant is released. The next rising edge can register a new grant if a request is pending. This gives a one-cycle gap between back-to-back packets from different masters.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

14 Packet Interleave Prevention

14.1 Rule

A slave can source data from only one master within a single packet. The grant is locked from the first beat of the packet until the TLAST beat is accepted.

14.2 Mechanism

The `packet_active[s]` flag is set when a new grant is made. While `packet_active[s]` is high:

- `grant_matrix[s]` is held constant.
- The crossbar mux continues to select the same master.
- `s_axis_tready` for the granted master follows only `m_axis_tready[s]`.

When `m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]` is true, `packet_active[s]` clears and the grant is released.

14.3 Why It Matters

AXI4-Stream slaves often expect packet-atomic streams. If beats from different masters were interleaved, a slave receiving the mixed stream could misinterpret packet boundaries, corrupt state, or violate protocol.

14.4 Example: Attempted Interleave

Suppose M0 and M1 both target S0 and both have multi-beat packets:

Interleave Prevention

Cycle	Without atomicity (hypothetical)	With Delta packet atomicity
0	M0 beat 0	M0 beat 0
1	M1 beat 0 (interleaved!)	M0 beat 1
2	M0 beat 1	M0 beat 2 (TLAST)
3	M1 beat 1 (TLAST)	M1 beat 0

Delta prevents the interleaved case by holding M0's grant until its `TLAST`.

14.5 Corner Cases

- If the granted master drops `TVALID` mid-packet, the slave sees no valid beats until the master reasserts it. The grant is still held.
- If the downstream slave is never ready, the packet can stall indefinitely; there is no timeout. This is consistent with standard AXI4-Stream behavior.

---

15 Module Structure

15.1 Generated File

The generator produces one file per configuration:

`delta_axis_flat_MxN.sv`

where `M` is `NUM_MASTERS` and `N` is `NUM_SLAVES`.

15.2 Module Header

The module begins with parameters and port declarations. The exact port list depends on the selected richness options.

15.2.1 Base Flavor Header

```
module delta_axis_flat_4x16 #(
    parameter int DATA_WIDTH  = 64,
    parameter int DEST_WIDTH  = 4,
    parameter int ID_WIDTH    = 2,
    parameter int USER_WIDTH  = 1,
    parameter int NUM_MASTERS = 4,
    parameter int NUM_SLAVES  = 16
)(
    input  logic aclk,
    input  logic aresetn,
    // s_axis and m_axis base ports...
);
```

15.2.2 Rich Flavor Additions

When `--strb-width` is used, `STRB_WIDTH` is added to the parameter list and `s_axis_tstrb`/`m_axis_tstrb` are added to the ports.

When `--amba5` is used, the check-width parameters `TDATACHK_W`, `TIDCHK_W`, `TDESTCHK_W` and `TUSERCHK_W` are added, plus `TSTRBCHK_W` with `--strb-width`. Each is derived from its payload width. The ports gain `s_/m_axis_twakeup`, the nine IHI 0051B check signals per side (`*_axis_tdatachk` … `*_axis_twakeupchk`), and the sticky `s_parity_error[NUM_MASTERS]` / `m_parity_error[NUM_SLAVES]`. The module then instantiates `rtl/common/dataint_parity`, so its filelist must `-f` include `rtl/common/filelists/dataint_parity.f`.

15.3 Internal Organization

Inside the module, the generator emits sections in this order:

1. Header comment with configuration.
2. Request decode.
3. Per-slave round-robin arbiter.
4. Crossbar data mux.
5. TREADY steering.
6. TWAKEUP steering (if enabled).
7. Interface check signals (if `--amba5`).
8. Performance counters (unless disabled).
9. Assertions (under `ifdef FORMAL`).

15.4 Filelist

The checked-in RTL includes `$REPO_ROOT`-relative filelists under `rtl/filelists/` for use with simulation and synthesis flows.

---

16 Parameters and Logic Implications

16.1 Parameter Summary

Parameter-to-Logic Mapping

Parameter	Generated From	Logic Implication
DATA_WIDTH	--data-width	Width of TDATA; with --amba5 it also sets TDATACHK_W = DATA_WIDTH/8.
DEST_WIDTH	clog2(NUM_SLAVES)	Width of TDEST; determines request decode comparison width.
ID_WIDTH	clog2(NUM_MASTERS)	Width of TID; pass-through only.
USER_WIDTH	--user-width	Width of TUSER; pass-through only.
NUM_MASTERS	--masters	Size of s_axis_* arrays and grant/request matrix columns.
NUM_SLAVES	--slaves	Size of m_axis_* arrays and grant/request matrix rows.
STRB_WIDTH	--strb-width	Width of TSTRB; adds mux ports when > 0.
TDATACHK_W, TSTRBCHK_W, TIDCHK_W, TDESTCHK_W, TUSERCHK_W	derived in the header from the payload widths (IHI 0051B Table 5-1)	Check-signal widths with --amba5. Each sizes one carried *CHK mux field and one bank of dataint_parity checkers per master. TSTRBCHK_W appears only with --strb-width.

16.2 Width Calculations

The generator computes:

```
dest_width = max(1, ceil(log2(num_slaves)))
id_width   = max(1, ceil(log2(num_masters)))
```

A single master or slave still gets a 1-bit width to avoid zero-width signals.

16.3 Parameter Interaction

- `STRB_WIDTH == 0` omits `TSTRB` entirely.
- The check-width parameters exist only when `--amba5` is set. Without it, no `TWAKEUP` or `*CHK` ports are generated.
- `--amba5` with a `--data-width` that is not a multiple of 8 is rejected by the CLI parser. The old `--parity-width` option has been removed, because the spec fixes the check widths.

16.4 RTL Constants

The generator uses `generate`/`genvar` loops sized by `NUM_MASTERS` and `NUM_SLAVES`. Changing these parameters after generation requires regenerating the module; the parameter is set at generation time, not truly configurable post-generation. This is intentional: it allows the synthesizer to optimize the fixed-size structure.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

17 Test Strategy

17.1 Current Status

As of 2026-10-12, Delta has one cocotb suite, `dv/tests/test_delta_axis_flat_2x4_strb_amba5.py`. It runs against the checked-in `--amba5` example through a flattening harness (`dv/tb/`), with RDS-DV AXIS5 BFMs (`enable_parity=True`) on all six ports and the TB class in `dv/tbclasses/delta_axis_amba5_tb.py`. Three tests run at gate, func and full: routing with every check signal verified (random VALID gaps and READY stalls at func and full), TWAKEUP steering, and check-error injection on every flag. The base flavor still has no simulation suite. The plan below remains the target for it.

17.2 Recommended Test Plan

When a testbench is created, it should cover the following areas.

17.2.1 1. Routing Coverage

- For every master `m` and every slave `s`, send at least one packet from `m` to `s`.
- Verify that only the target slave receives the packet.
- Verify that `m_axis_tdest[s]` matches the sent `TDEST`.

17.2.2 2. Arbitration Fairness

- Have multiple masters send continuous packets to the same slave.
- Verify that each master receives approximately equal bandwidth over time.
- Verify that no master is starved.

17.2.3 3. Packet Atomicity

- Have two masters send multi-beat packets to the same slave.
- Verify that slave receives the first packet's beats contiguously up to `TLAST` before any beat of the second packet.

17.2.4 4. Backpressure

- Randomly deassert `m_axis_tready[s]`.
- Verify that only the granted master stalls and that no data is lost.

17.2.5 5. Reset

- Assert `aresetn` mid-transaction.
- Verify all grants clear and counters reset.

17.2.6 6. Rich Flavor

- With `--strb-width`, verify `TSTRB` pass-through.
- With `--amba5`, verify `TWAKEUP` OR-per-slave, the odd-parity check signals (payload `*CHK` carried, `TVALIDCHK`/`TWAKEUPCHK`/`TREADYCHK` generated or checked) and the sticky `*_parity_error` flags. The existing suite already covers this.

17.2.7 7. Formal Assertions

- Run with `+define+FORMAL` or equivalent.
- Verify the one-hot grant and TDEST-range assertions never fire.

17.3 Lint and Generation Checks

For the base flavor, which has no testbench yet, verification consists of:

```
python3 bin/delta_generator.py --topology flat --masters 4 --slaves 16
\
    --data-width 64 --output-dir rtl/
verilator --lint-only rtl/delta_axis_flat_4x16.sv
```

The planned mesh cell verification suites are summarized in Chapter 6, Section 8.

---

18 Buffers and Virtual Channels

IMPLEMENTED: This chapter describes the `delta_mesh_cell` microarchitecture as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

18.1 Per-Port Input Buffer Organization

Each `delta_mesh_cell` has nine AXI4-Stream input ports: six axis neighbor ports (±X, ±Y, ±Z), one diagonal tetrad port (P_DIA), one diagonal tile port (P_DIA2), and one local port. P_DIA and P_DIA2 are the two instances of the single conceptual diagonal axis. Every input port keeps two virtual channels, so the buffering resource is `9 ports × 2 VCs`. Each VC is a physically separate buffer partition with its own single-port readout; the two partitions on one port share only the incoming flit stream up to the VC demux.

Input buffer organization

Resource	Count per cell	Notes
Input ports	9	6 axis + P_DIA + P_DIA2 + 1 local
VCs per port	2	VC0 = escape, VC1 = adaptive/QoS alternate
Buffer partitions	18	one single-port readout array per VC
Read ports consumed	9	one per input port, multiplexing its two VCs into the pipeline

The single-port-per-input choice is deliberate. It is cheaper than a fully-ported VC-per-input arrangement, and it matches the expected throughput because only one flit per input port advances in a given cycle. The real cost is area, not bandwidth: Proteo reported that FIFO flops dominated a NoC node at 81% of area [N8: §4.5], so the generator exposes a regs-vs-SRAM buffer knob.

18.2 Why Per-VC Reservation Is Required

A shared input buffer that any VC can draw from looks efficient until adaptive routing starts bouncing packets. Here is the part that bites: a packet can hold the last free slot in a shared buffer while waiting for a credit that is held by another packet, which in turn is waiting for a slot in the same buffer. That cycle is a deadlock, and it is exactly why the literature insists on per-VC buffer reservation for VC-based deadlock avoidance. Duato's sufficient condition for deadlock freedom assumes that a packet queued on one channel cannot block resources belonging to another channel [D2][D7]; Dally & Towles restate the same requirement for per-VC input buffer organization [P18: §17.1]. Partitioning the buffer keeps the channel-dependency graph clean and makes the escape-channel proof hold.

> **Design note:** A shared read port is fine; a shared *storage pool* is not. The demux that writes a flit into VC0 or VC1 must be deterministic from the flit's VC field; do not let a full VC0 spill into VC1 space, or the deadlock analysis collapses.

18.3 Credit-Based Flow Control

Every (output port, VC) pair maintains a credit counter that tracks the number of free buffer slots at the downstream input VC. A flit may leave this cell only if the corresponding credit is non-zero. When the downstream cell consumes the flit, it returns a credit. The counter depth must cover the round-trip latency so the channel can run at full duty factor:

```
F · Lf / tcrt ≥ b
```

where `F` is the flow-control frequency, `Lf` is the forward channel latency, `tcrt` is the credit-return latency, and `b` is the minimum number of credits needed to keep the pipe full. If the inequality fails, the link will go idle waiting for credits even though both ends have data [P10][P15].

Credit counter sizing

Counter	Location	Width driver
Downstream credit	Per (output port, VC)	round-trip latency × flits per cycle
Consumed credit	Per (output port, VC)	same as credit counter

18.4 Failure Mode: Ready Drops Mid-Burst

So what happens when TREADY drops mid-burst on an output? The flit stays in the upstream buffer, the credit is not returned, and the upstream pipeline stalls for that VC only. Because each VC has its own credit counter, a backpressured VC1 path does not block VC0 escape traffic. That isolation is the whole point of splitting the buffers.

18.5 References

- Duato, Yalamanchili, Ni, *Interconnection Networks: An Engineering Approach*, 2nd ed., Morgan Kaufmann, 2003 — channel-dependency theory and VC-based deadlock avoidance [D2][D7].
- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004 — per-VC input buffer reservation and credit sizing [P10][P15][P18].
- Jantsch & Tenhunen (eds.), *Networks on Chip*, Kluwer, 2003 — Proteo node area and FIFO cost data [N8: §4.5].

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

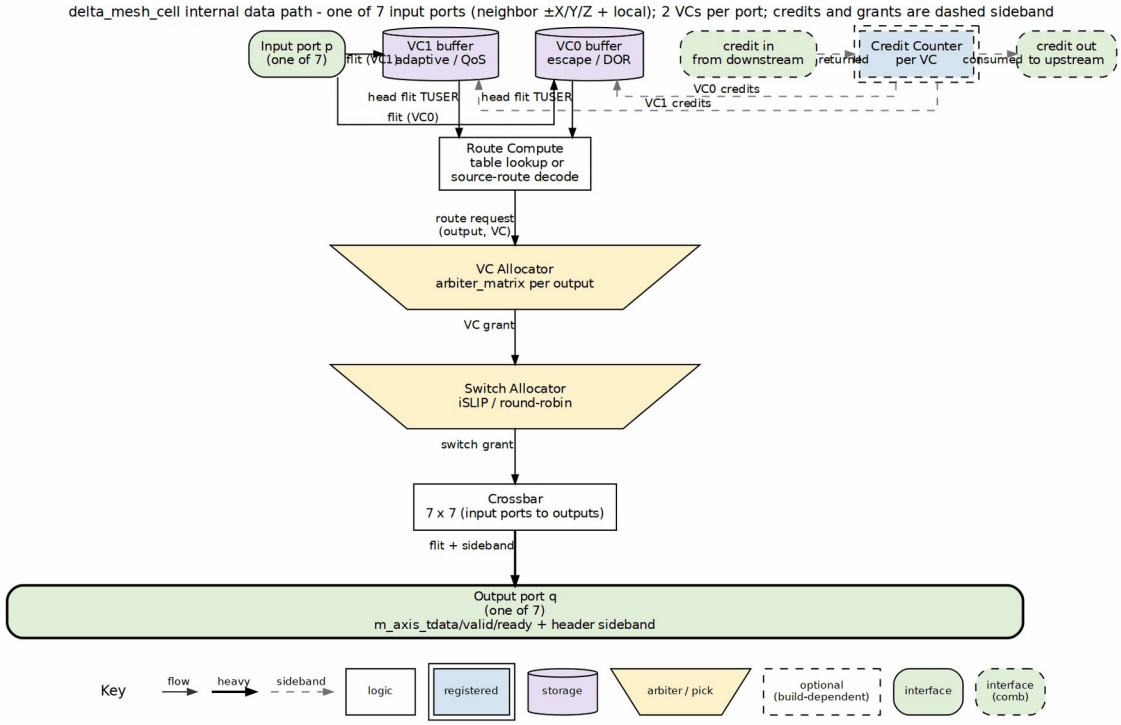
19 Pipeline Stages

IMPLEMENTED: This chapter describes the `delta_mesh_cell` pipeline as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

The cell pipeline is organized as five stages. Each stage is a valid/ready skid so a downstream stall does not corrupt state.

19.0.1 Figure 6.1: `delta_mesh_cell` internal data path

19.0.2 Figure 6.1: `delta_mesh_cell` internal data path



Cell pipeline stages

Stage	Function	Cycle-critical notes
Buffer write	Demux arriving flit into VC0 or VC1 partition	Only head flit carries the route header
Route compute	Table lookup or source-route decode; produce (output port, VC)	Lookahead pre-computes the *next* hop's output port
VC allocation	Pick an available downstream VC for the	Separable per-output matrix arbiters

Stage	Function	Cycle-critical notes
	chosen output	
Switch allocation	Pick a crossbar input for each output port	iSLIP or round-robin; one-hot grant
Crossbar traversal	Mux the winning flit onto the output port	Combinational data path, registered valid

19.1 Route Lookahead

The steady-state target is one cycle per hop. That only works if the route decision for hop n+1 is computed while the flit is still traversing hop n. The head flit carries enough sideband in TUSER to let the next cell determine its own output port without an extra lookup cycle [P17]. Body flits follow the same path until TLAST releases the switch grant.

Measured latency on the ring and tetrad suites is 4 cycles/hop today (issue #120). The 1 cycle/hop steady-state target is deferred; the current pipeline pays the full table-lookup and grant latency per hop. Source-route mode can bypass the table lookup for fixed-latency systolic schedules.

19.2 Speculative VC and Switch Allocation

The generator can enable speculative allocation: a head flit requests both a VC and a switch slot before the route is fully known, overlapping the allocators with the lookup. If the route guess is wrong, the grant is discarded and the flit retries the next cycle. The steady-state benefit is largest under light load; under saturation, mis-speculation wastes allocator cycles [P17]. The flag is per-build (`--speculation`), not dynamic.

19.3 Handshake Discipline

A granted input holds its output until the packet's TLAST handshake completes, identical in spirit to the flat crossbar's packet-atomic arbitration. The switch allocator releases the crossbar input only after `TLAST && TVALID && TREADY`. Body beats advance on every ready cycle; if TREADY falls mid-burst, the beat waits in the output stage and the grant remains held.

19.4 Failure Mode: TLAST Arrives Without a Matching Grant

That should never happen. If it does, the cell is in an inconsistent state: the flit has no output port and will be dropped. The verification suite includes a formal assertion that `TLAST` may only be asserted while a switch grant is active for the same input, mirroring the flat-mode packet-atomicity check.

19.5 References

- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004 — router speculation and lookahead [P17].

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 Routing Table

IMPLEMENTED: This chapter describes the `delta_mesh_cell` routing table as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

20.1 RAM Table Structure

Each cell contains a RAM lookup table. The key is the packet's destination coordinate plus its QoS class; the entry gives the output port mask and the VC assignment. The mask supports multicast on VC0 and unicast on either VC.

Routing table entry layout

Field	Width	Purpose
Key: `dest_x`, `dest_y`, `dest_z`	`3 × COORD_W`	Destination coordinate, sized by fabric dimensions
Key: QoS class	2	0=config, 1=latency, 2=bulk
Entry: output port mask	9 bits (one per output port)	Next-hop direction(s); one-hot for unicast, multi-hot for multicast
Entry: VC	1	0 or 1, assigned by the table

20.2 Reset Image

At reset the table is loaded with a dimension-ordered route toward the rapids boundary. The route is minimal, connected, and acyclic by construction, which satisfies Duato's sufficient condition for the escape VC [D5][P11]. This bootstrap path is the only way to load custom tables in-band; every other mode image is loaded later through the config port.

20.3 In-Band Load and Readback

Configuration packets target a reserved address. The ejection block intercepts them instead of delivering them to the local port. The command format is `{op, entry_index, entry_data}`; reads return response packets to the source for load-verify. Config traffic rides QoS class 0 on the reset-default route. A boundary cell or rapids DMA injects the stream.

> **Design note:** The generator can also emit preloaded ROM images for the standard fabric modes. In that build the cell still uses the same table RAMs; the image is just initialized differently.

20.4 VC0 Acyclicity Check

VC0 is the escape channel. Its table entries must form a connected, acyclic subrelation in the extended channel-dependency graph; that is Duato's sufficient condition for deadlock freedom [D2][D7]. This check is performed *offline* on a table image before it is loaded, not by the hardware at runtime. The verification target is the table-load flow: any image that fails the acyclicity test is rejected before it reaches the fabric.

20.5 Torus Wrap and the Dateline Rule

When a dimension is configured as a torus (`--torus=x,y,z`), the wrap link creates a cycle that plain dimension-order routing cannot break. The fix is a dateline: one designated wrap channel per dimension participates in VC0 only as part of the escape subrelation; ordinary wrap traffic uses VC1 and may consume a misroute budget [P11]. The hardware enforces the rule by gating wrap-link VC0 grants with the dateline condition.

20.6 Standard Fabric Modes

Alongside custom per-cell tables, the generator defines ten standard modes switched by a broadcast mode-switch message. Each cell holds generator-emitted images; the switch applies only after a fabric-wide ack barrier returns through the config read-response path.

Standard fabric mode catalog

Mode	Pattern	Routing source
1	Mesh (default)	RAM tables, reset DOR image
2	Systolic-1D	Fixed source-route paths
3	Wavefront-2D	Per-diagonal static tables
4	Stencil-3D	Nearest-neighbor tables + torus wrap
5	Broadcast	Multicast VC0 port masks to all
6	Convergecast/reduce	Reverse-broadcast tables
7	All-to-all/transpose	Per-stage table images
8	Pipeline + rapids-stream	Fixed chain + boundary tables
9	Tetrad all-to-all (K4)	P_DIA for diagonal mates; DOR for inter-tetrad dsts
10	Tile all-to-all (K4 of tetrads)	P_DIA2 for diagonal-tetrad mates; DOR for inter-tile dsts

20.7 References

- Duato, Yalamanchili, Ni, *Interconnection Networks: An Engineering Approach*, 2nd ed., Morgan Kaufmann, 2003 — deterministic routing, Duato's protocol, and VC0 acyclicity [D2][D5][D7].
- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004 — table-based routing, deadlock avoidance, and the dateline technique [P8][P11][P12].
- Franklin et al. (eds.), *Network Processor Design: Issues and Practices, Vol. 1*, Morgan Kaufmann, 2002 — in-band control-program precedent [Q6].

21 Allocators

IMPLEMENTED: This chapter describes the `delta_mesh_cell` allocators as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

The cell reuses the house arbiters rather than inventing new allocation hardware. The two allocator stages are VC allocation and switch allocation.

Allocator mapping

Allocator	House block	Role
VC allocation	`arbiter_matrix`	For each output port, pick one requesting input VC that has a free downstream VC
Switch allocation	Round-robin or iSLIP-style	For each output port, pick one input flit that already won a VC

21.1 VC Allocation

VC allocation is a many-to-one pick per output port: several input VCs request the same downstream VC, and the arbiter grants at most one. The `arbiter_matrix` is Dally & Towles' recommended choice for small input counts [P19: §18.5]. It keeps state in a priority matrix, provides fairness, and is small enough to place one per output port without exploding area. The grant is one-hot per output.

> **Design note:** A separable allocator would allocate VC and switch in one combined stage. That saves a cycle but at the cost of more hardware and lower saturation throughput. The two-stage separable approach is the conservative starting point; combined allocation is deferred until measurements justify it.

21.2 Switch Allocation

Switch allocation picks which input port drives each output port. The baseline is a round-robin arbiter per output. An iSLIP-style allocator is selectable at generation; iSLIP's iterative accept/reject loop reaches 100% throughput under uniform traffic, while a one-pass separable allocator saturates around 66% [P19: §19.3, §19.7]. The choice is a throughput-vs-area knob.

21.3 One-Hot Grant Assertions

Every allocator grant is asserted to be one-hot (or zero-hot). Under `ifdef FORMAL`, a `$onehot0` assertion checks each output's grant vector. This is the same discipline as the flat crossbar: a multi-hot grant means two inputs would collide on the same output.

21.4 Multicast Allocation

Multicast entries use a multi-hot output port mask. The cell does not allocate all outputs at once; instead it incrementally serves whichever masked outputs are free each cycle and releases the input only after all requested outputs have been served [P18: §17.2][P19: exercises]. Until then the head flit stays at the front of its input VC and the body flits behind it cannot advance. That serialization is the price of source-side replication within one cell.

21.5 Conservative Output-VC Reallocation

When a downstream VC drains, the cell waits for the returning credit before assigning that VC to a new packet. Conservative reallocation prevents a subtle failure: if a VC were reassigned before the credit returned, the new packet could establish a dependency that the original deadlock analysis did not cover [P16]. Waiting one round trip is cheap insurance.

21.6 References

- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004 — matrix arbiters, separable and iSLIP allocators, and multicast allocation [P16][P18][P19].

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

22 QoS Mechanisms

IMPLEMENTED: This chapter describes the `delta_mesh_cell` QoS mechanisms as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Delta defines three QoS classes with strict priority at every output port. The split follows the control/data-plane separation common in NPU practice: class 0 for config, class 1 for latency traffic, class 2 for bulk [Q4][Q5].

QoS class mapping

Class	Name	Priority	Typical use
0	Config / control	Highest	In-band table loads, mode switches, rapids control
1	Latency	Middle	Compute traffic with deadline requirements
2	Bulk	Lowest	Background data movement

22.1 Strict Priority and Programmable Aging

The arbiter uses strict priority by default: a class-0 request always wins over class 1 and 2. To prevent indefinite starvation of lower classes, each class has a programmable aging counter. After the counter expires, the effective priority inverts so the lower class gets service [D13]. The counter is per-output-port and reloads automatically.

22.2 Bulk Minimum Reservation

Aging alone is not enough. Under sustained latency traffic, bulk could still be starved because every fresh class-1 request would preempt the aged class-2 grant. The cell therefore maintains a bulk minimum reservation counter: after a programmable number of consecutive higher-priority grants, the next grant is forced to class 2 regardless of pending higher-priority requests [D1][Q5]. This guarantees a floor of bulk bandwidth without relying on traffic patterns.

22.3 Class Non-Interference

Priority bits are insufficient when traffic classes share buffers. Tree-saturation in one class can block unrelated traffic through a shared input buffer even if that traffic has a priority bit [P14: §15.5]. Delta avoids this by giving each class its own VC *and* its own injection buffer at the local port. Config traffic rides a dedicated slice of VC0 so control can never be bulk-blocked.

Class isolation resources

Isolation mechanism	Where applied
Separate VCs per class	Every input port
Separate injection buffers per class	Local port only
Independent table region per class	Per-destination route diversity

22.4 Failure Mode: Aging Inversion Misconfiguration

If the aging counter is set too large, class 2 starves. If it is set too small, class 1 loses its latency advantage. The verification suite sweeps the counter and checks that bulk gets at least its reserved fraction under a tree-saturation attack.

22.5 References

- Duato, Yalamanchili, Ni, *Interconnection Networks: An Engineering Approach*, 2nd ed., Morgan Kaufmann, 2003 — starvation under priority and bulk reservation need [D1][D13].
- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004 — separation of resources to avoid tree saturation [P14].
- Franklin et al. (eds.), *Network Processor Design: Issues and Practices, Vol. 1*, Morgan Kaufmann, 2002 — NPU QoS practice and ASPEN scheduler starvation prevention [Q4][Q5].

23 Multicast Path

IMPLEMENTED: This chapter describes the `delta_mesh_cell` multicast path as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Multicast is restricted to VC0. The routing table supplies a multi-hot output port mask, and the cell builds a BRCP-conformed path: every branch follows the base unicast routing function so multicast adds no new channel dependencies beyond the unicast proof [D9]. The implementation floods a multicast packet along a BFS tree from the source, replicating at branching cells.

23.1 Source-Side Replication

When a head flit reaches a cell whose table entry says “send out ports A and B,” the cell creates one copy per destination output. This is source-side replication at the branching cell, not a broadcast-and-filter scheme. It costs buffer slots at the branch but avoids wasting downstream bandwidth on flits that would be dropped [Q7].

23.2 Delivery-VC Assignment

Multicast *reception* uses multiple delivery VCs at each destination cell. The literature is clear: even with correct routing, a single delivery channel for multicast creates a deadlock that the unicast VC analysis does not cover [D9: §5.5.3]. The cell assigns a distinct delivery VC per multicast group or per source, depending on the build-time parameterization.

23.3 The No-Responses Rule

Multicast connections do not carry responses. If every destination tried to acknowledge delivery, the acknowledgments would converge on the source and create a many-to-one merge problem; worse, a response could wait for a multicast channel that is held by another response, closing a dependency cycle [N3]. Many-to-one communication uses unicast, not multicast.

Multicast discipline

Rule	Rationale
VC0 only	Keeps multicast on the escape channel; avoids adaptive-route cycles
BRCP paths	Adds no channel dependencies beyond unicast proof
Source-side replication	No wasted downstream bandwidth
Multiple delivery VCs	Breaks delivery-channel deadlock
No responses	Avoids response merge and cyclic waits

23.4 Failure Mode: Tree-Based Multicast in Wormhole Networks

A naive tree multicast that forks at every hop in a wormhole network deadlocks readily because forked branches can hold buffer resources in a cycle [D9: §5.5.2]. The BRCP + source replication + delivery-VC combination is the book-recommended discipline for this reason.

23.5 References

- Duato, Yalamanchili, Ni, *Interconnection Networks: An Engineering Approach*, 2nd ed., Morgan Kaufmann, 2003 — hardware multicast deadlock, BRCP, and delivery-channel VCs [D9].
- Jantsch & Tenhunen (eds.), *Networks on Chip*, Kluwer, 2003 — iEthereal connection model: multicast with no responses [N3].
- Franklin et al. (eds.), *Network Processor Design: Issues and Practices, Vol. 1*, Morgan Kaufmann, 2002 — source-side replication precedent [Q7].

24 Observability Counters and DFT

IMPLEMENTED: This chapter describes the `delta_mesh_cell` counter bank and DFT mode as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

24.1 Ocelus Counter Bank

The cell exposes eleven 32-bit saturating counters through the config-read path. The counter bank is selected by setting `table_index[15] = 1`; indices 0..10 are defined and indices above 10 read back 0. Counters are frozen when `ENABLE_COUNTERS == 0` and clear on `aresetn`.

Ocelus counter bank indices.

Index	Name	What it counts
0	C_FLITS_IN	Flits received by the cell
1	C_FLITS_OUT	Flits transmitted by the cell
2	C_PKTS_IN	Packets received (`TLAST` handshakes)
3	C_PKTS_OUT	Packets transmitted (`TLAST` handshakes)
4	C_STALL_CYCLES	Output stall cycles
5	C_CREDIT_STALL	Cycles a granted output waited for downstream credit
6	C_ERR_DROP	Malformed-destination drops (mirrors `o_err_drop_cnt`)
7	C_MODE_ACKS	Broadcast mode-switch acks received (mirrors `o_mode_acks`)
8	C_BULK_STARVE	Cycles bulk reservation was due
9	C_SPEC_COMMITS	Speculative grants that committed
10	C_SPEC_REVOKES	Speculative grants that were revoked

The suite is ahead of the NoC textbook literature, which says little about hardware observability, and is instead modeled on NPU practice [Q4][Q2][Q8].

24.2 DFT Test Mode

DFT mode reconfigures each cell into one of three deterministic routing patterns so the mesh wiring behaves as a concatenated bus. Stuck-at and bridge faults become testable with approximately `ceil(log2(n))` patterns, where `n` is the number of cells in the fabric [N6]. The mode is entered through a DFT escape write on the config port (`table_index[15:12] = 4'hD`, payload bit 0 enables DFT, bits 2:1 select the pattern); each cell drives an `o_dft_active` output.

DFT routing patterns

Pattern	Routing behavior	Fault target
Pattern 0	All cells route +X	Horizontal wiring
Pattern 1	All cells route +Y	Vertical wiring
Pattern 2	All cells route +Z	Depth wiring

> **Design note:** DFT mode is mutually exclusive with normal traffic. The cell gates non-test flits while `test_active` is asserted; returning to normal mode requires a fabric-wide mode-switch sequence.

24.3 References

- Jantsch & Tenhunen (eds.), *Networks on Chip*, Kluwer, 2003 — testing the communication infrastructure with three switch configurations [N6].
- Franklin et al. (eds.), *Network Processor Design: Issues and Practices, Vol. 1*, Morgan Kaufmann, 2002 — IXP2400 PMU, Toaster per-queue stats, Agere ASI RMON counters [Q4][Q2][Q8].

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

25 Verification Targets

IMPLEMENTED: This chapter describes the `delta_mesh_cell` verification suites as built. The design is per the 2026-10-13 design spec (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

25.1 cocotb Suites

The mesh verification plan is ten cocotb suites, each driven by a different generator configuration.

Mesh cocotb verification suites

Suite	Focus	Pass criteria
Cell routing	Reset DOR, programmed tables, config load + readback	Correct output port per table; readback matches write
Fabric 2x2	All-pairs delivery across multi-hop paths	Every packet delivers exactly once
Flow control	Credit exhaustion, atomicity, steady-state throughput	No loss, no interleaving, sustained 1 flit/clk
Tables	Malformed-dst drop, cross-VC drop drain, table reroute	Drop counted once; drain stays on its own VC
Modes	Broadcast mode-switch, ack barrier, mode images 1/2/3/5/6	Barrier completes; traffic routes per selected image
Tetrad	Mode 9 K4 all-to-all, P_DIA one-hop latency	All 12 ordered pairs deliver in 1 hop
Tile	Mode 10 K4-of-tetrads, P_DIA2 one-hop latency	Diagonal-tetrad pairs deliver in 1 hop
QoS	Strict priority, aging inversion, bulk reservation	Class order under saturation; bulk minimum bandwidth
Multicast	Fanout correctness, no-response rule, deadlock soak	All destinations receive exactly one copy; 10^6-flit soak green
Counters	Counter bank readback, DFT escape	Readback matches scripted traffic; DFT pattern switches
Systolic	Torus ring, dateline, source-route, speculation	Ring latency measured; dateline forces VC0; SR bypasses table drop;

Suite	Focus	Pass criteria
		speculation parity
Perf model calibration	Tornado pattern at three loads vs `bin/delta_mesh_perf.py`	Zero-load T0 within 20% of model; loaded deltas recorded

25.2 Formal Assists

Three formal checks assist the cocotb suites:

1. **VC0 acyclicity checker.** Run offline on table images before load; rejects any VC0 subrelation that violates Duato's sufficient condition [D2].
2. **One-hot grant assertions.** Per-output switch and VC grants must be one-hot or zero-hot, identical to the flat-mode assertion discipline.
3. **TLAST-before-grant-release packet atomicity.** A grant may not be released before the `TLAST` handshake completes, preventing packet interleaving on an output.

25.3 Performance Model Calibration

The Python performance model (`bin/delta_mesh_perf.py`) predicts zero-load latency and saturation throughput for uniform, nearest-neighbor, tornado, bit-complement, and transpose patterns. The perf-model suite compares measured results against the model and reports deltas; the model itself is verified by the same methodology used for network-processor benchmarks [P20][Q1].

25.4 References

- Duato, Yalamanchili, Ni, *Interconnection Networks: An Engineering Approach*, 2nd ed., Morgan Kaufmann, 2003 — acyclicity condition for deadlock-free escape VC [D2].
- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004 — performance analysis methodology [P20].
- Franklin et al. (eds.), *Network Processor Design: Issues and Practices, Vol. 1*, Morgan Kaufmann, 2002 — hardware-level benchmarking practice [Q1].